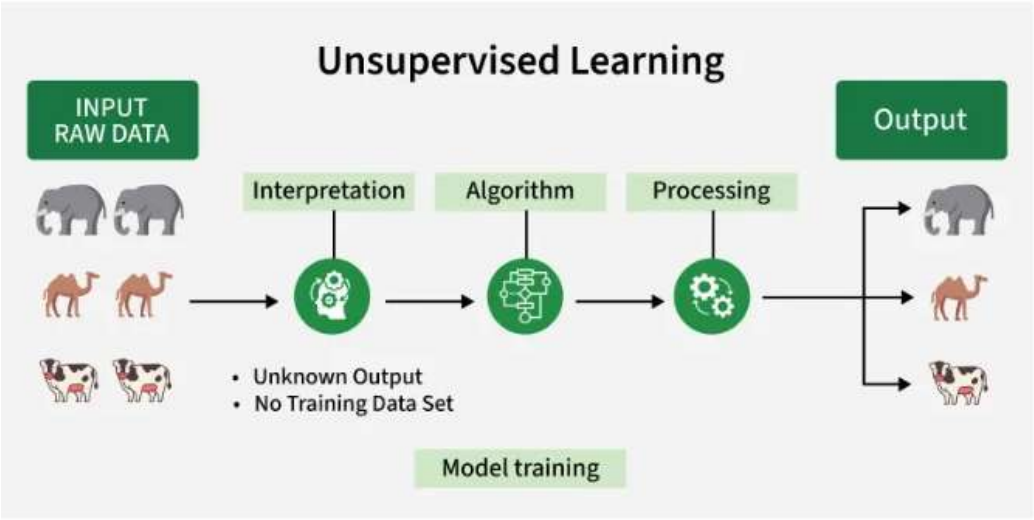
# What is Unsupervised Learning?

- 
- 

Unsupervised Learning is a type of machine learning where the model works without labelled data. It learns patterns on its own by grouping similar data points or finding hidden structures without any human intervention.

- It is used for tasks like clustering, dimensionality reduction and Association Rule Learning.
- Helps identify hidden patterns in data
- Useful for grouping, compression and anomaly detection




**Unsupervised Learning**
The image shows set of animals like elephants, camels and cows that represents raw data that the unsupervised learning algorithm will process.

- The "Interpretation" stage signifies that the algorithm doesn't have predefined labels or categories for the data. It needs to figure out how to group or organize the data based on inherent patterns.
- An algorithm represents unsupervised learning process that helps to identify patterns in the data.
- The processing stage shows the algorithm working on the data.

The output shows the results of the unsupervised learning process. In this case, the algorithm might have grouped the animals into clusters based on their species (elephants, camels, cows).

**Working of Unsupervised Learning**

The working of unsupervised machine learning can be explained in these steps:

**1. Collect Unlabeled Data**

- Gather a dataset without predefined labels or categories.
- **Example**: Images of various animals without any tags.

**2. Select an Algorithm**

- Choose a suitable unsupervised algorithm such as clustering like K-Means, association rule learning like Apriori or dimensionality reduction like PCA based on the goal.

**3. Train the Model on Raw Data**

- Feed the entire unlabeled dataset to the algorithm.
- The algorithm looks for similarities, relationships or hidden structures within the data.

**4. Group or Transform Data**

- The algorithm organizes data into groups (clusters), rules or lower-dimensional forms without human input.
- Example: It may group similar animals together or extract key patterns from large datasets.

**5. Interpret and Use Results**

- Analyze the discovered groups, rules or features to gain insights or use them for further tasks like visualization, anomaly detection or as input for other models.

**Unsupervised Learning Algorithms**

There are mainly 3 types of Unsupervised Algorithms that are used:

**1. Clustering Algorithms**

Clustering is an unsupervised machine learning technique that groups unlabeled data into clusters based on similarity. Its goal is to discover patterns or relationships within the data without any prior knowledge of categories or labels.

- Groups data points that share similar features or characteristics.
- Helps find natural groupings in raw, unclassified data.
- Commonly used for customer segmentation, anomaly detection and data organization.
- Works purely from the input data without any output labels.
- Enables understanding of data structure for further analysis or decision-making.

Some common clustering algorithms:

- **K-means Clustering:** Groups data into K clusters based on how close the points are to each other.
- **Hierarchical Clustering:** Creates clusters by building a tree step-by-step, either merging or splitting groups.
- **Density-Based Clustering (DBSCAN):** Finds clusters in dense areas and treats scattered points as noise.
- **Mean-Shift Clustering:** Discovers clusters by moving points toward the most crowded areas.

- **Spectral Clustering:** Groups data by analyzing connections between points using graphs.

**2. Association Rule Learning**

Association rule learning is a rule-based unsupervised learning technique used to discover interesting relationships between variables in large datasets. It identifies patterns in the form of "if-then" rules, showing how the presence of some items in the data implies the presence of others.

- Finds frequent item combinations and the rules connecting them.
- Commonly used in market basket analysis to understand product purchase relationships.
- Helps retailers design promotions and cross-selling strategies.

Some common Association Rule Learning algorithms:

- **Apriori Algorithm:** Finds patterns by exploring frequent item combinations step-by-step.
- **FP-Growth Algorithm:** An Efficient Alternative to Apriori. It quickly identifies frequent patterns without generating candidate sets.
- **Eclat Algorithm:** Uses intersections of itemsets to efficiently find frequent patterns.
- **Efficient Tree-based Algorithms:** Scales to handle large datasets by organizing data in tree structures.

**3. Dimensionality Reduction**

Dimensionality reduction is the process of decreasing the number of features or variables in a dataset while retaining as much of the original information as possible. This technique helps simplify complex data making it easier to analyze and visualize. It also improves the efficiency and performance of machine learning algorithms by reducing noise and computational cost.

- It reduces the dataset's feature space from many dimensions to fewer, more meaningful ones.
- Helps focus on the most important traits or patterns in the data.
- Commonly used to improve model speed and reduce overfitting.

Here are some popular Dimensionality Reduction algorithms:

- **Principal Component Analysis (PCA):** Reduces dimensions by transforming data into uncorrelated principal components.
- **Linear Discriminant Analysis (LDA):** Reduces dimensions while maximizing class separability for classification tasks.
- **Non-negative Matrix Factorization (NMF):** Breaks data into non-negative parts to simplify representation.
- **Locally Linear Embedding (LLE)**: Reduces dimensions while preserving the relationships between nearby points.
- **Isomap**: Captures global data structure by preserving distances along a manifold.

**Applications of Unsupervised learning**

Unsupervised learning has diverse applications across industries and domains. Key applications include:

- **Customer Segmentation**: Algorithms cluster customers based on purchasing behavior or demographics, enabling targeted marketing strategies.
- **Anomaly Detection**: Identifies unusual patterns in data, aiding fraud detection, cybersecurity and equipment failure prevention.
- **Recommendation Systems**: Suggests products, movies or music by analyzing user behavior and preferences.
- **Image and Text Clustering**: Groups similar images or documents for tasks like organization, classification or content recommendation.
- **Social Network Analysis**: Detects communities or trends in user interactions on social media platforms.

**Advantages**

- **No need for labeled data:** Works with raw, unlabeled data hence saving time and effort on data annotation.
- **Discovers hidden patterns**: Finds natural groupings and structures that might be missed by humans.
- **Handles complex and large datasets**: Effective for high-dimensional or vast amounts of data.
- **Useful for anomaly detection**: Can identify outliers and unusual data points without prior examples.

**Challenges**

Here are the key challenges of unsupervised learning:

- **Noisy Data**: Outliers and noise can distort patterns and reduce the effectiveness of algorithms.
- **Overfitting Risk**: Overfitting can occur when models capture noise instead of meaningful patterns in the data.
- **Limited Guidance**: The absence of labels restricts the ability to guide the algorithm toward specific outcomes.
- **Cluster Interpretability**: Results such as clusters may lack clear meaning or alignment with real-world categories.

# Expectation-Maximization

The **Expectation-Maximization (EM) algorithm** is a powerful iterative optimization technique used in unsupervised machine learning to estimate unknown parameters in statistical models, especially when the data contains unobserved **latent variables** or is incomplete.

It solves a "chicken-and-egg" problem: to find the optimal model parameters, you need the hidden data; to find the hidden data, you need the model parameters. EM alternates between guessing the hidden data and refining the model until it converges to

### Core Mechanism: The Two-Step Cycle

The algorithm operates through a repeating cycle of two main phases after an initial random guess of parameters.

1. **Expectation Step (E-Step):**

- **Estimate Hidden Data:** Using current parameter guesses, the algorithm calculates the expected values or probabilities for the latent variables.
- **"Soft" Assignment:** Instead of strictly assigning a data point to one category, it assigns "responsibilities"—probabilities that a point belongs to a particular hidden group.

2. **Maximization Step (M-Step):**

- **Update Parameters:** The algorithm treats the estimated values from the E-step as if they were real data and recalculates the model parameters (like mean or variance) to maximize the likelihood of the observed data.
- **Improvement:** Each M-step is mathematically guaranteed to never decrease the model's likelihood.

The cycle repeats until the change in parameters becomes negligible, a state known as **convergence**

# K means Clustering – Introduction

K-Means Clustering groups similar data points into clusters without needing labeled data. It is used to uncover hidden patterns when the goal is to organize data based on similarity.

- Helps identify natural groupings in unlabeled datasets
- Works by grouping points based on distance to cluster centers
- Commonly used in customer segmentation, image compression, and pattern discovery

- Useful when you need structure from raw, unorganized data

**Working of K-Means Clustering**

Suppose we are given a data set of items with certain features and values for these features like a vector. The task is to categorize those items into groups. To achieve this we will use the K-means algorithm. "k" represents the number of groups or clusters we want to classify our items into.

The algorithm will categorize the items into "k" groups or clusters of similarity. To calculate that similarity we will use the Euclidean distance as a measurement. The algorithm works as follows:

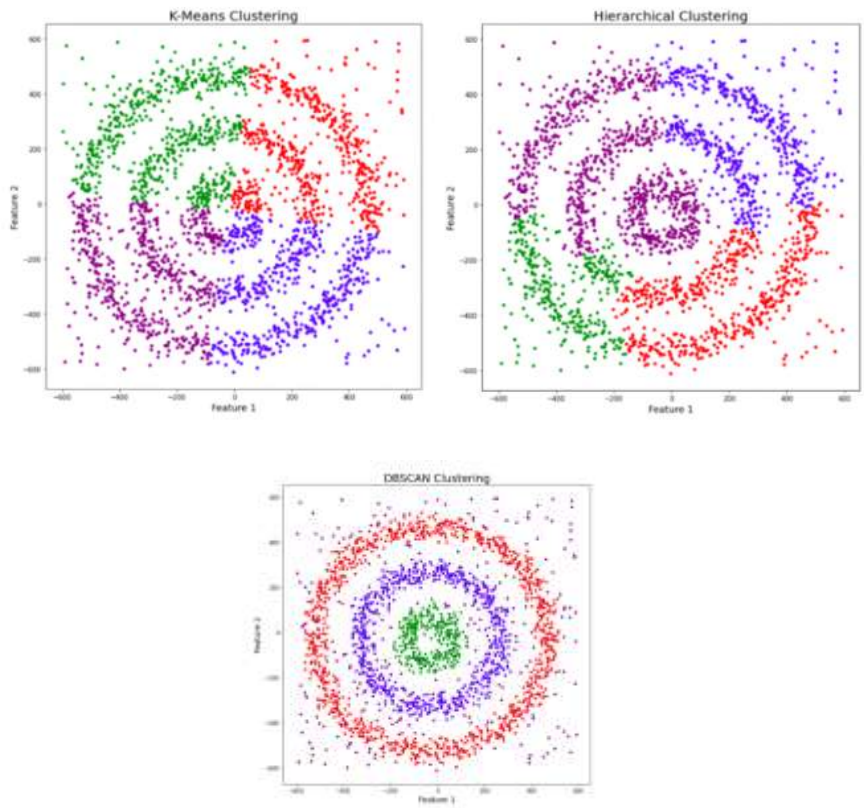
1. **Initialization:** We begin by randomly selecting k cluster centroids.
2. **Assignment Step:** Each data point is assigned to the nearest centroid, forming clusters.
3. **Update Step:** After the assignment, we recalculate the centroid of each cluster by averaging the points within it.

**Repeat:** This process repeats until the centroids no longer change or the maximum number of iterations is reached.

# DBSCAN Clustering in ML - Density based clustering

DBSCAN is a density-based clustering algorithm that groups data points that are closely packed together and marks outliers as noise based on their density in the feature space. It identifies clusters as dense regions in the data space separated by areas of lower density. Unlike K-Means or hierarchical clustering which assumes clusters are compact and spherical, DBSCAN perform well in handling real-world data irregularities such as:

- **Arbitrary-Shaped Clusters**: Clusters can take any shape not just circular or convex.
- **Noise and Outliers**: It effectively identifies and handles noise points without assigning them to any cluster.




**Data Sets with clustering algorithms**

The figure above shows a data set with clustering algorithms: K-Means and Hierarchical handling compact, spherical clusters with varying noise tolerance while DBSCAN manages arbitrary-shaped clusters and noise handling.

**Key Parameters in DBSCAN**

**1. eps**: This defines the radius of the neighborhood around a data point. If the distance between two points is less than or equal to eps they are considered neighbors. A common method to determine eps is by analyzing the k-distance graph. Choosing the right eps is important:

- If eps is too small most points will be classified as noise.
- If eps is too large clusters may merge and the algorithm may fail to distinguish between them.

**2. MinPts**: This is the minimum number of points required within the **eps** radius to form a dense region. A general rule of thumb is to set MinPts >= D+1 where **D** is the number of dimensions in the dataset.

*For most cases a minimum value of **MinPts = 3** is recommended.*

**How Does DBSCAN Work**

DBSCAN works by categorizing data points into three types:

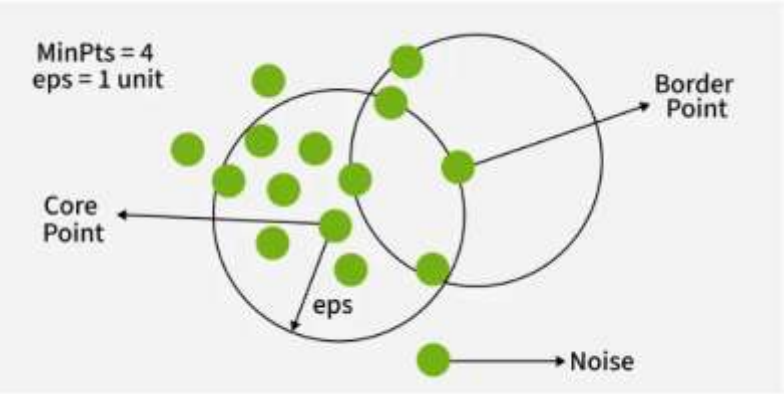
1. Core points which have a sufficient number of neighbors within a specified radius (eplison)
2. Border points which are near core points but lack enough neighbors to be core points themselves
3. Noise points which do not belong to any cluster.

By iteratively expanding clusters from core points and connecting density-reachable points, DBSCAN forms clusters without relying on rigid assumptions about their shape or size.

# DBSCAN ALgorithm

**Steps in the DBSCAN Algorithm**

1. **Identify Core Points**: For each point in the dataset count the number of points within its eps neighborhood. If the count meets or exceeds MinPts mark the point as a core point.
2. **Form Clusters**: For each core point that is not already assigned to a cluster create a new cluster. Recursively find all density-connected points i.e points within the eps radius of the core point and add them to the cluster.
3. **Density Connectivity**: Two points a and b are density-connected if there exists a chain of points where each point is within the eps radius of the next and at least one point in the chain is a core point. This chaining process ensures that all points in a cluster are connected through a series of dense regions.
4. **Label Noise Points**: After processing all points any point that does not belong to a cluster is labeled as noise.




DBSCAN Algorithm

# EM algorithm

In clustering, the EM algorithm is most commonly implemented through Gaussian Mixture Models (GMM), where it performs **soft clustering**—assigning each data point a probability of belonging to each cluster rather than a single fixed label.

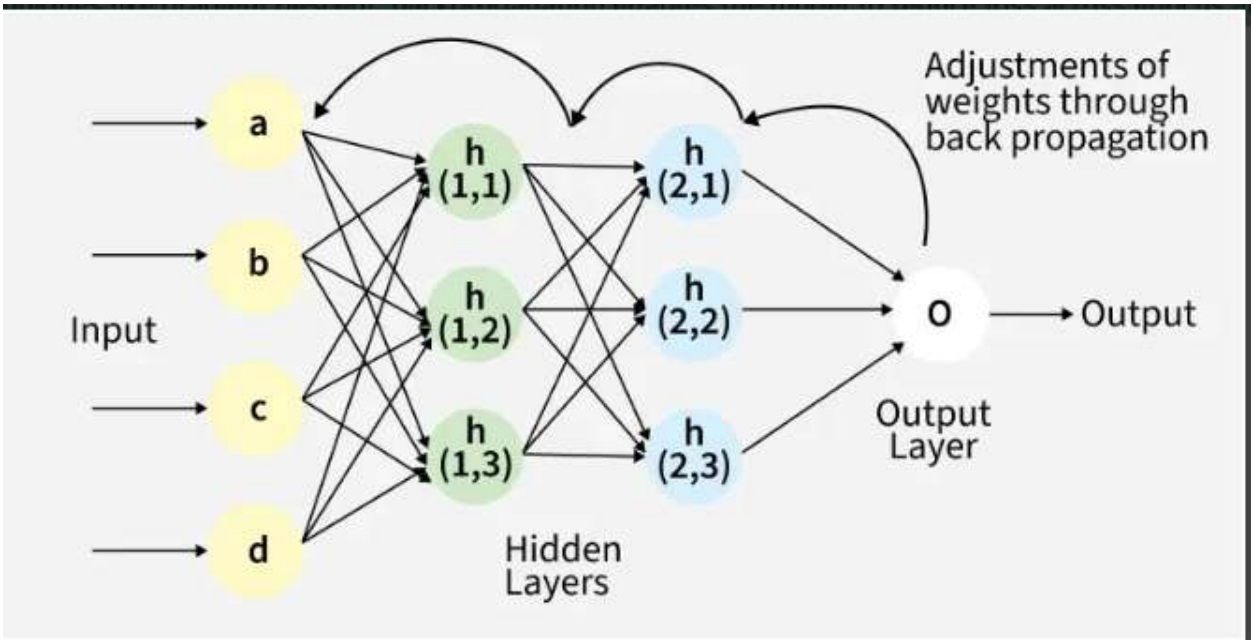
The Expectation-Maximization (EM) algorithm is an iterative optimization method used to find maximum likelihood estimates of parameters in probabilistic models with hidden or missing (latent) variables. It alternates between calculating the expected values of latent variables (E-step) and updating parameters to maximize this log-likelihood (M-step) until convergence.

**Steps of the EM Algorithm**

1. **Initialize:** Start with an initial guess for the parameter values ().
2. **E-Step (Expectation):** Calculate the expected value of the latent variables ("missing data") based on the current parameter estimates, effectively determining the "responsibilities" for each data point.
3. **M-Step (Maximization):** Update the model parameters () by maximizing the expected log-likelihood found in the E-step.
4. **Iterate:** Repeat steps 2 and 3 until the parameters converge (no longer change significantly).

# Backpropagation in Neural Network

Backpropagation, short for Backward Propagation of Errors, is a key algorithm used to train neural networks by minimizing the difference between predicted and actual outputs. It works by propagating errors backward through the network, using the chain rule of calculus to compute gradients and then iteratively updating the weights and biases. Combined with optimization techniques like gradient descent, backpropagation enables the model to reduce loss across epochs and effectively learn complex patterns from data.




Fig(a) A simple illustration of how the backpropagation works by adjustments of weights

Back Propagation plays a critical role in how neural networks improve over time. Here's why:

1. **Efficient Weight Update**: It computes the gradient of the loss function with respect to each weight using the chain rule making it possible to update weights efficiently.
2. **Scalability**: The Back Propagation algorithm scales well to networks with multiple layers and complex architectures making deep learning feasible.
3. **Automated Learning**: With Back Propagation the learning process becomes automated and the model can adjust itself to optimize its performance.

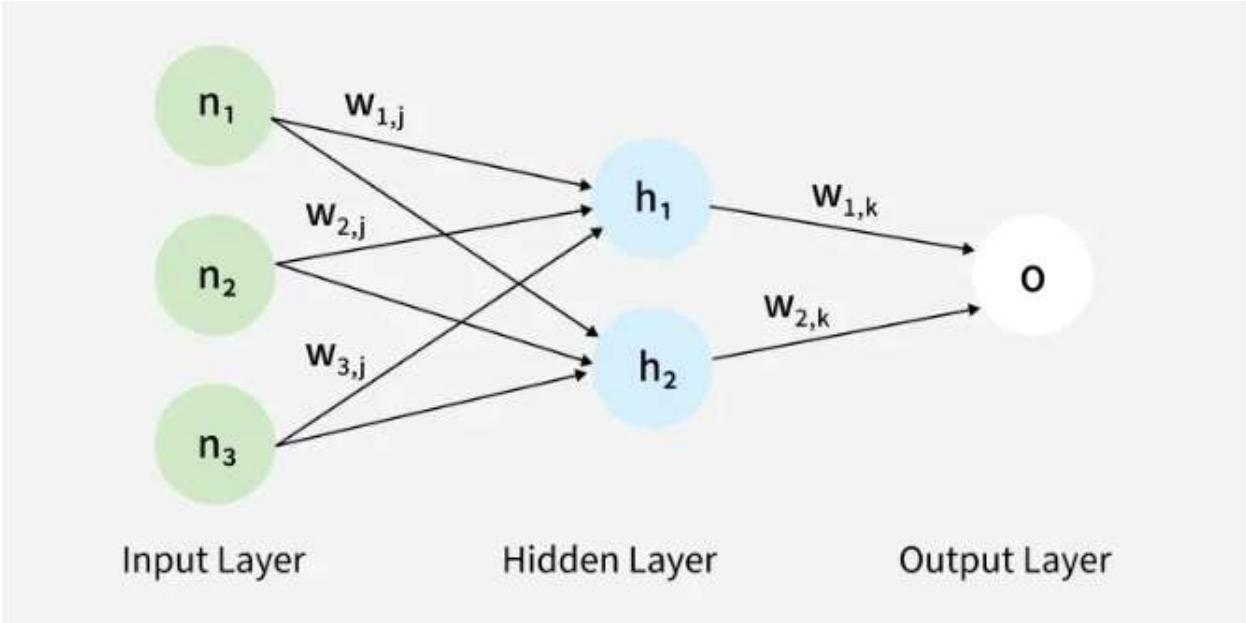
**Working of Back Propagation Algorithm**

The Back Propagation algorithm involves two main steps: the Forward Pass and the Backward Pass.

**1. Forward Pass Work**

In forward pass the input data is fed into the input layer. These inputs combined with their respective weights are passed to hidden layers. For example in a network with two hidden layers (h1 and h2) the output from h1 serves as the input to h2. Before applying an activation function, a bias is added to the weighted inputs.

Each hidden layer computes the weighted sum (`a`) of the inputs then applies an activation function like ReLU (Rectified Linear Unit) to obtain the output (`o`). The output is passed to the next layer where an activation function such as softmax converts the weighted outputs into probabilities for classification.

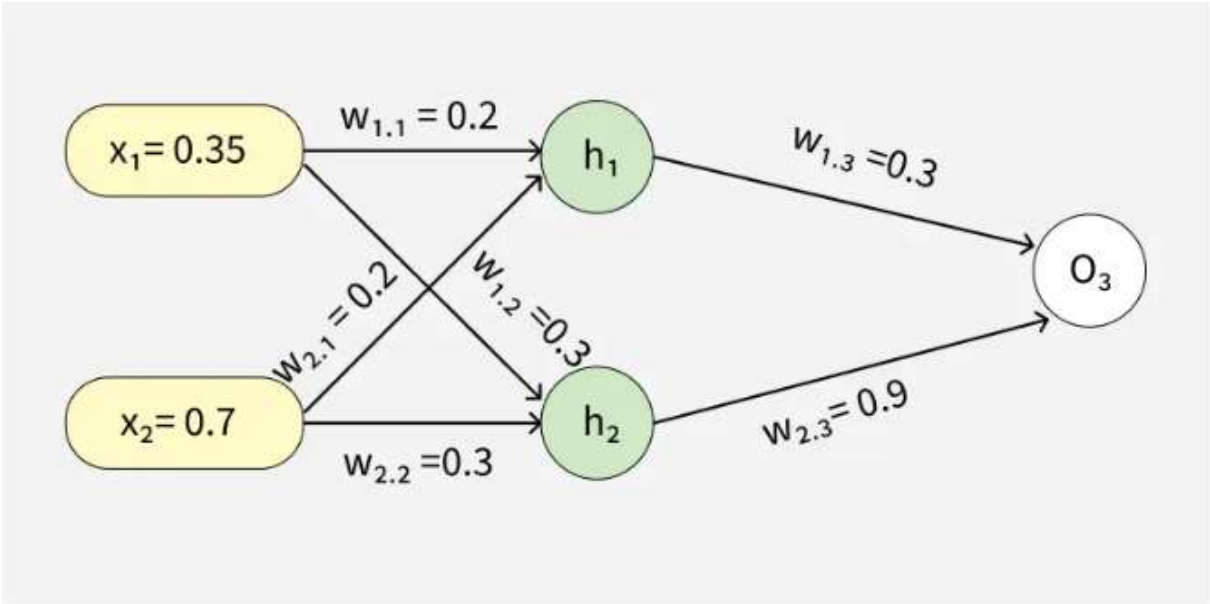



**2. Backward Pass**

In the backward pass the error (the difference between the predicted and actual output) is propagated back through the network to adjust the weights and biases. One common method for error calculation is the Mean Squared Error (MSE) given by:

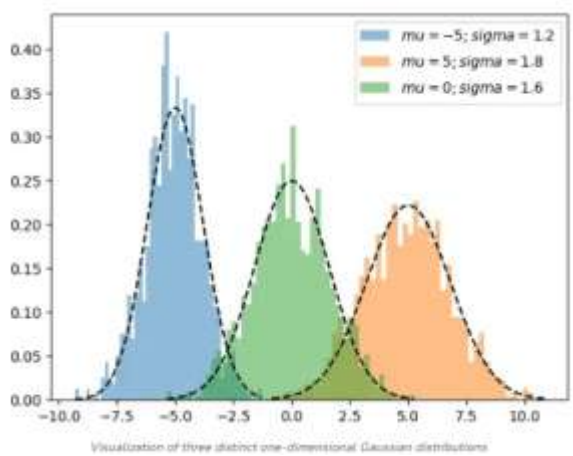
$$\text{MSE} = (\text{Predicted Output} - \text{Actual Output})^2$$

Once the error is calculated the network adjusts weights using gradients which are computed with the chain rule. These gradients indicate how much each weight and bias should be adjusted to minimize the error in the next iteration. The backward pass continues layer by layer ensuring that the network learns and improves its performance. The activation function through its derivative plays a crucial role in computing these gradients during Back Propagation.




# Gaussian Mixture Model

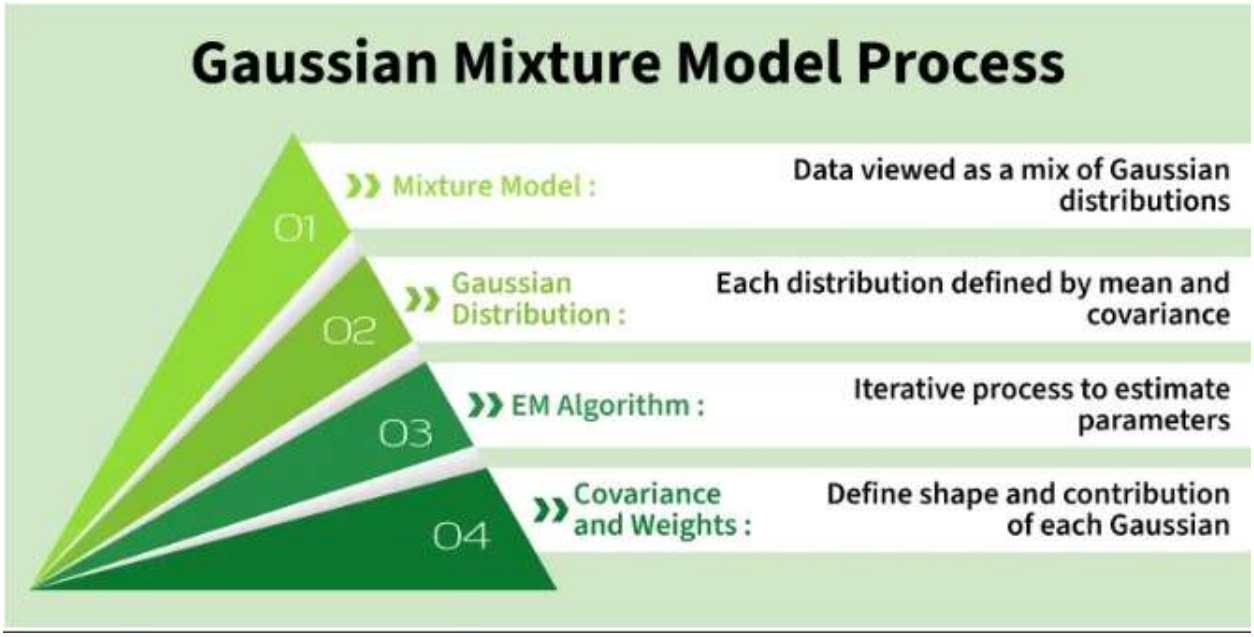
A Gaussian Mixture Model (GMM) is a probabilistic model that assumes data points are generated from a mixture of several Gaussian (normal) distributions with unknown parameters. Unlike hard clustering methods such as K-Means which assign each point to a single cluster based on the closest centroid, GMM performs soft clustering by assigning each point a probability of belonging to multiple clusters.




Visualization of three distinct one-dimensional Gaussian distributions

The above shown graph shows a three one-dimensional Gaussian distributions with distinct means and variances. Each curve represents the theoretical probability density function (PDF) of a normal distribution, highlighting differences in location and spread.

**Working of GMM**




Working of Gaussian Mixture Model

A Gaussian Mixture Model assumes that the data is generated from a mixture of K Gaussian distributions, each representing a cluster. Every Gaussian has its own mean μk$\mu k$, covariance Σk$\Sigma k$ and mixing weight πk$\pi k$.

**1. Posterior Probability (Cluster Responsibility)**

For a given data point $x_n$, GMM computes the probability that it belongs to cluster k:

$$P(z_n = k \mid x_n) = \frac{\pi_k \cdot \mathcal{N}(x_n|\mu_k, \Sigma_k)}{\sum_{k-1}^{K} \pi_k \cdot \mathcal{N}(x_n|\mu_k, \Sigma_k)}$$

where:

- $z_n$ is a latent variable indicating which Gaussian the point belongs to.
- $\pi_k$ is the mixing probability of the k-th Gaussian.
- $\mathcal{N}(x_n \mid \mu_k, \Sigma_k)$is the Gaussian distributio n with mean $\mu_k$ and covariance $\Sigma_k$

**2. Likelihood of a Data Point**

The total likelihood of observing xnx_nxn under all Gaussians is:

$$P(x_n) = \sum_{k-1}^{K} \pi_k \mathcal{N}(x_n \mid \mu_k, \Sigma_k)$$

This represents how well the mixture as a whole explains the data point.

### 3. Expectation-Maximization (EM) Algorithm

GMMs are trained using the EM algorithm, an iterative process that estimates the best parameters:

**E-step (Expectation)**: Compute the responsibility of each cluster for every data point using current parameter values.

**M-step (Maximization):**Update

- Means $\mu_k$
- Covariances $\Sigma_k$
- Mixing coefficients $\pi_k$

using the responsibilities from the E-step. The process continues until the model's log-likelihood stabilizes.

### 4. Log-Likelihood of the Mixture Model

The objective optimized by EM is:

$$L(\mu_k, \Sigma_k, \pi_k) = \prod_{n-1}^{N} \sum_{k-1}^{K} \pi_k \mathcal{N}(x_n \mid \mu_k, \Sigma_k)$$

EM increases this likelihood in every iteration.

# Role of GMM

The **Gaussian Mixture Model (GMM)** plays a significant role in Machine Learning, especially in **unsupervised learning and probabilistic modeling**. It is a flexible model used to represent data as a combination of multiple Gaussian (normal) distributions.

## 1. Clustering (Soft Clustering)

- GMM is widely used for clustering tasks.
- Unlike hard clustering methods like K-means clustering, GMM performs **soft clustering**.
- Each data point is assigned a **probability of belonging to each cluster**, rather than a fixed label.

## 2. Modeling Complex Data Distributions

- Real-world data often does not follow a single distribution.
- GMM models data as a **mixture of multiple Gaussian distributions**, making it suitable for:
    - Multi-modal datasets
    - Overlapping clusters

## 3. Density Estimation

- GMM is used to estimate the **probability density function (PDF)** of a dataset.
- Helps in understanding how data points are distributed.

### 4. Anomaly Detection

- By learning the normal data distribution, GMM can identify **outliers**.
- Data points with very low probability under the model are considered anomalies.

### 5. Data Generation

- Once trained, GMM can generate **new synthetic data samples** from learned distributions.
- Useful in simulations and generative modeling.

### 6. Expectation-Maximization (EM) Algorithm

- GMM uses the Expectation-Maximization algorithm for parameter estimation.
- EM iteratively:
    - Estimates cluster probabilities (E-step)
    - Updates Gaussian parameters (M-step)

### 7. Feature Representation & Dimensionality Reduction

- GMM can be used to represent data in terms of **probability distributions**, improving feature extraction in some ML pipelines.

# Hard Margin and Soft Margin in SVM

In Support Vector Machine (SVM), the concepts of **hard margin** and **soft margin** define how the algorithm handles classification boundaries, especially in relation to data separability and noise.

---

## 🔹 1. Hard Margin SVM

### ✔ Definition

A **hard margin SVM** is used when the data is **perfectly linearly separable**. It finds a decision boundary (hyperplane) that separates classes **without any misclassification**.

### ✔ Key Idea

- No tolerance for errors
- All data points must lie **outside the margin boundary**

### ✔ Mathematical Condition

For each data point:

$$y_i(w \cdot x_i + b) \geq 1$$

### ✔ Characteristics

- Maximizes margin strictly
- No slack variables
- Works only for **clean, noise-free data**

### ✔ Limitations

- Not suitable for real-world data (which often contains noise)
- Very sensitive to outliers

## 🔹 2. Soft Margin SVM

### ✔ Definition

A **soft margin SVM** allows **some misclassification** to handle non-separable or noisy data.

### ✔ Key Idea

- Introduces **slack variables ($\xi_i$)** to allow violations
- Balances between **maximizing margin** and **minimizing classification error**

### ✔ Mathematical Formulation

$$\min \frac{1}{2}||w||^2 + C\sum_{i-1}^{n}\xi_i \quad \text{subject to } y_i(w \cdot x_i + b) \geq 1 - \xi_i,\ \xi_i \geq 0$$

### ✔ Characteristics

- Allows some points inside margin or misclassified
- Controlled by parameter **C**:
  - Large C → fewer errors, smaller margin
  - Small C → more errors allowed, larger margin

### ✔ Advantages

- Works well with **real-world, noisy datasets**
- More flexible and practical

---

## 🔹 Key Differences

| Feature | Hard Margin SVM | Soft Margin SVM |
|---|---|---|
| Data Requirement | Perfectly separable | Non-separable / noisy |
| Error Tolerance | No errors allowed | Some errors allowed |
| Flexibility | Rigid | Flexible |
| Use of Slack Variables | No | Yes |
| Real-world Use | Rare | Common |

Role of Activation Function in ML

Activation functions are essential in **Artificial Neural Network (ANNs)** because they introduce **non-linearity**, enabling the network to learn complex patterns beyond simple linear relationships.

# Role of Activation Functions

- Transform input signals into output signals
- Introduce **non-linearity**
- Help in learning complex data patterns
- Control the flow of information in the network

## 🔹 1. ReLU (Rectified Linear Unit)

### ✔ Definition

The **Rectified Linear Unit (ReLU)** outputs:

$$f(x) = \max(0, x)$$

### ✔ Role

- Converts negative values to **zero** and keeps positive values unchanged
- Speeds up training of deep networks
- Reduces vanishing gradient problem

### ✔ Advantages

- Simple and computationally efficient
- Widely used in deep learning

### ✔ Limitation

- "Dead neuron" problem (neurons stop learning if output is always 0)

## 2. Sigmoid Function

### Definition

The **Sigmoid function maps input to:

$$f(x) = \frac{1}{1 + e^{-x}}$$

### Role

- Converts input into a **probability value (0 to 1)**
- Commonly used in **binary classification output layers**

### Advantages

- Smooth and differentiable
- Output interpretable as probability

### Limitations

- Suffers from **vanishing gradient problem**
- Output not zero-centered

## 3. Tanh Function

### Definition

The **Hyperbolic tangent function outputs:

$$f(x) = \tanh(x)$$

### Role

- Maps input to range **(-1 to 1)**
- Centers data around zero, improving learning

### Advantages

- Better than sigmoid for hidden layers
- Zero-centered output

### Limitations

- Still suffers from **vanishing gradient problem**

### 🔹 Comparison Table

| Feature | ReLU | Sigmoid | Tanh |
|---|---|---|---|
| Output Range | [0, ∞) | (0, 1) | (-1, 1) |
| Non-linearity | Yes | Yes | Yes |
| Vanishing Gradient | Less | High | Moderate |
| Speed | Fast | Slow | Moderate |
| Common Use | Hidden layers | Output layer (binary) | Hidden layers |

# Hidden Markov Model (HMM)

The **Hidden Markov Model (HMM)** is a powerful probabilistic model used to represent systems where the underlying states are **hidden (not directly observable)** but can be inferred through observable outputs.

# 1. What is an HMM?

An HMM is a **doubly stochastic process** consisting of:

- A sequence of **hidden states**
- A sequence of **observations** generated from those states

Example: In speech recognition, the spoken words (states) are hidden, but the audio signals (observations) are visible.

## 🔹 2. Components of HMM

An HMM is defined by five elements:

### ✔️ 1. Set of Hidden States (S)

$$S = \{s_1, s_2, \ldots, s_N\}$$

These are not directly observable.

### ✔ 2. Observation Symbols (V)

$$V = \{v_1, v_2, ..., v_M\}$$

These are visible outputs.

### ✔ 3. Transition Probability Matrix (A)

$$A = \{a_{ij}\}, \quad a_{ij} = P(s_j \mid s_i)$$

Represents probability of moving from one state to another.

### ✔ 4. Emission Probability Matrix (B)

$$B = \{b_j(k)\}, \quad b_j(k) = P(v_k \mid s_j)$$

Probability of observing symbol $v_k$ in state $s_j$.

### ✔ 5. Initial State Distribution (π)

$$\pi = \{\pi_i\}, \quad \pi_i = P(s_i \text{ at } t = 1)$$

## 🔹 3. Complete Model Representation

An HMM is represented as:

$$\lambda = (A, B, \pi)$$

---

## 🔹 4. Working of HMM

1. Start in an initial state based on π
2. Transition between states using matrix A
3. Generate observations using matrix B
4. Continue the process over time steps

The **Max-Sum** and **Max-Product** algorithms are inference techniques used in probabilistic graphical models such as **Bayesian Network** and **Markov Random Field**. They are message-passing algorithms that operate on **factor graphs** to compute optimal or most probable values.

# 1. Max-Product Algorithm

### ✓ Definition

The **Max-Product algorithm** is used to find the **most probable configuration (MAP: Maximum A Posteriori)** of variables.

### ✓ Key Idea

- Messages are passed between nodes
- Uses **multiplication of probabilities** and **maximization**

### ✔ Message Update Rule

For node $x_i$ sending a message to factor $f$:

$$m_{x_i \to f}(x_i) = \prod_{h \in \mathrm{ne}(x_i) \setminus f} m_{h \to x_i}(x_i)$$

For factor $f$ to variable $x_j$:

$$m_{f \to x_j}(x_j) = \max_{x_{\sim j}} \left[ f(x) \prod_{i \neq j} m_{x_i \to f}(x_i) \right]$$

### ✔ Role

- Finds **most likely assignment of variables**
- Used in decoding problems like:
  - HMM (via **Viterbi algorithm**)
  - Error-correcting codes

## 🔹 2. Max-Sum Algorithm

### ✔ Definition

The **Max-Sum algorithm** is a **logarithmic version** of the Max-Product algorithm.

### ✔ Key Idea

- Converts multiplication into addition using log:

$$\log(ab) = \log a + \log b$$

- Avoids numerical underflow

### ✔ Message Update Rule

$$m_{f \to x_j}(x_j) = \max_{x_{\sim j}} \left[ \log f(x) + \sum_{i \neq j} m_{x_i \to f}(x_i) \right]$$

### ✔ Role

- Used for **optimization problems**
- More stable for computation
- Widely used in:
  - Computer vision
  - Energy minimization problems